

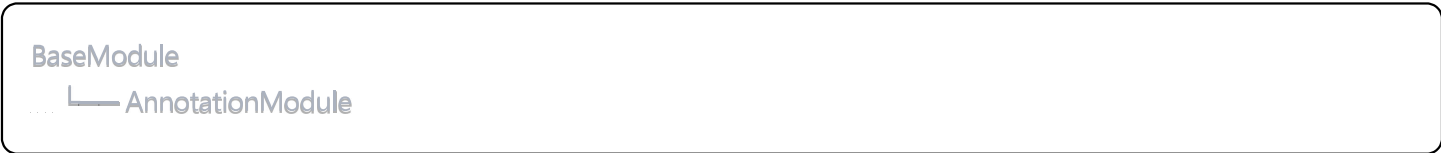
AnnotationModule - Developer Documentation v2.0

Overview

The refactored `AnnotationModule` provides enhanced image annotation functionality for the IMPACT Editor system. This module allows users to create, edit, and manage annotations on images within documents, with improved error handling, performance, and maintainability.

Architecture

Class Hierarchy



Key Components

- **Core Module:** Main annotation management
- **Dimension Calculator:** Responsive sizing logic
- **Event Handler:** Annotation lifecycle management
- **Error Handler:** Simplified error tracking
- **Legacy Compatibility:** Backward compatibility layer

Dependencies

Required Global Objects

javascript

```

// Core Editor Dependencies
GlobalEditor..... // Main editor instance
CKEDITOR..... // CKEditor library
NewQueryModule    // Query and DOM manipulation
IMPACT_SELECTION  // Editor selection state
EDITOR_CURSOR..... // Cursor position tracking

// UI Dependencies
$..... // jQuery library
SweetAlert2..... // Alert dialogs (optional)

// Utility Functions
debug.log()..... // Debug logging
ErrorLogTrace()   // Common error logging
s4()..... // Unique ID generation
GET_JSON()..... // JSON data preparation

// Configuration
IS_LOCAL_HOST..... // Development flag
DOC_ROOT_SELECTOR..... // Document root selector

```

Plugin Dependencies

```

javascript

// CKEditor Plugin Path
CKEDITOR.plugins.getPath('ImageAnotation_old')

```

Class Structure

Constructor

```

javascript

constructor(name, errorTracker, options = {})

```

Parameters:

- `name` (string): Module identifier
- `errorTracker` (object): Error tracking instance
- `options` (object): Configuration options

Initialization Sequence:

1. Call parent constructor
2. Initialize properties and state
3. Bind methods to preserve context
4. Setup constants and selectors
5. Mark as initiated

Public Methods

Core Functionality

`initLoop()`

Initializes the annotation processing loop.

```
javascript  
  
initLoop()  
// Sets: AutoInitiated = true, FullyLoaded = true, isModuleReady = true
```

`showLoop(options = {})`

Main entry point to display annotation interface.

```
javascript  
  
await showLoop(  
  ... // Configuration options  
)
```

Process Flow:

1. Validate prerequisites
2. Cache editor references
3. Setup UI elements
4. Extract image source data
5. Calculate optimal dimensions
6. Load annotation interface
7. Update editor image sources

fireRestore()

Restores annotation state (placeholder for restoration logic).

reset()

Resets module to initial state.

```
javascript
```

```
reset()
```

```
// Clears: iframe, references, state flags, annotation handler
```

closeModule()

Closes annotation interface and cleans up resources.

Dimension Management

setupDimensions()

Enhanced version of original `setUpWidthHeight()`.

```
javascript
```

```
setupDimensions()
```

```
// Calculates responsive dimensions and applies to image/body
```

Plugin Integration

loadPluginScript(annotationLibrary)

Loads and configures annotation plugin.

```
javascript
```

```
loadPluginScript(annotationLibrary)
```

```
// Extracts existing annotations, applies dimensions, loads annotations
```

bindAnnotationEvents(annotationLibrary)

Binds all annotation event handlers.

```
javascript
```

```
bindAnnotationEvents(annotationLibrary)
// Binds: created, updated, removed, mouseOver events
```

Utility Methods

`getUrl()`

Returns annotation interface URL with cache-busting.

```
javascript

const url = getUrl()
// Returns: "path/ImgAnnot.html?_ =timestamp"
```

Private Methods

Validation

- `validatePrerequisites()`: Checks required dependencies
- `checkUserPermissions()`: Validates user edit rights

UI Management

- `assignVariablesEventLoop()`: Maps UI selectors to elements
- `cacheEditorReferences()`: Caches editor objects for performance
- `applyFrameDimensions()`: Sets iframe dimensions
- `applyImageDimensions()`: Sets image element dimensions

Image Processing

- `getImageSource()`: Extracts image data from editor selection
- `findImageElement()`: Locates image element in DOM
- `updateEditorImageSources()`: Refreshes image sources in editor

Annotation Processing

- `extractExistingAnnotations()`: Gets existing annotations from document
- `parseAnnotationData()`: Parses annotation data from DOM
- `buildAnnotationDataString()`: Creates annotation data string
- `createAnnotationCommand()`: Generates DOM command for annotation

- `insertAnnotationIntoDocument()`: Inserts annotation into document
- `renumberAnnotations()`: Updates annotation numbering

Event Handlers

- `handleAnnotationCreated()`: Processes new annotations
- `handleAnnotationUpdated()`: Processes annotation updates
- `handleAnnotationRemoved()`: Processes annotation removal
- `handleBeforeAnnotationRemoved()`: Pre-removal processing
- `handleMouseOverAnnotation()`: Mouse interaction handling

Calculation

- `calculateOptimalDimensions()`: Computes frame dimensions
- `calculateResponsiveDimensions()`: Computes responsive sizing

Legacy Compatibility

Deprecated Methods (with warnings):

```
javascript
```

```
AssignVar_EventLoop() → assignVariablesEventLoop()
getSource()..... → getImageSource()
setUpWidthHeight()..... → setupDimensions()
pluginLoadScript() → loadPluginScript()
annotationEvtBind()..... → bindAnnotationEvents()
```

State Management

Module State Properties

```
javascript
```

```
{
... // Initialization flags
... isModuleReady: boolean,
... isFrameLoaded: boolean,
... Initiated: boolean,
... AutoInitiated: boolean,
... FullyLoaded: boolean,

... // Current context
... currentFigure: jQuery|null,
... currentElement: jQuery|null,
... currentArray: Array|null,
... annotationHandler: Object|null,

... // Cached references
... _editorRefs: {
...     impactSelection: Object,
...     editorCursor: Object,
...     globalEditor: Object
... }
}
```

Configuration State

```
javascript
```

```
_state: {  
  templateList: {  
    spinner: string // Loading spinner HTML  
  },  
  dimensions: {  
    minWidth: 300,  
    minHeight: 200,  
    maxProcessableHeight: 700,  
    defaultPadding: 150,  
    resizePercentage: 20  
  },  
  selectors: {  
    figureClass: '.fig',  
    graphicImage: '.graphic img',  
    commentSpan: 'span[data-class="ckcommentsfull"]',  
    annotateAttr: 'data-annotate',  
    commentBoxAttr: 'data-user-comment-box',  
    userNameAttr: 'data-user-name'  
  }  
}
```

Data Formats

Annotation Data Structure

javascript

```

// DOM Storage Format
annotationData = "x,y,width,height,src"
// Example: "0.15,0.25,0.30,0.20,image.jpg"

// Runtime Object Format
annotation = {
  ... src: "image.jpg",
  ... text: "Annotation text",
  ... shapes: [{
    ... type: 'rect',
    ... geometry: {
      ... x: 0.15, ... // Relative x position (0-1)
      ... y: 0.25, ... // Relative y position (0-1)
      ... width: 0.30, // Relative width (0-1)
      ... height: 0.20 // Relative height (0-1)
    }
  }]
}

```

Image Data Array

```

javascript

imageData = [
  ... [imageSrc], ... // ["path/to/image.jpg"]
  ... [imageWidth], ... // [800]
  ... [imageHeight] ... // [600]
]

```

Integration Points

CKEditor Integration

```

javascript

// Selection handling
const selection = GlobalEditor.getSelection();
const startElement = selection.getStartElement();

// DOM manipulation
const element = GlobalEditor.find('[data-annotate="${data}"]');
element.getParent().remove();

```

NewQueryModule Integration

```
javascript

// DOM insertion
const command = NewQueryModule.M_FUN.InsertDOM(text, 0, s4(), 'imgnote', 'new');

// Renumbering
NewQueryModule.M_FUN.ReNumberNote(GlobalEditor);
```

Error Tracking Integration

```
javascript

// Simple error logging
ErrorLogTrace('ANNOTATION_METHOD_NAME', error.message);
```

Static Utilities

AnnotationModule.Utls

validateAnnotationData(annotationData)

Validates annotation data format.

```
javascript

const isValid = AnnotationModule.Utls.validateAnnotationData("0.1,0.2,0.3,0.4,src");
// Returns: boolean
```

generateUniqueld()

Creates unique annotation identifier.

```
javascript

const id = AnnotationModule.Utls.generateUniqueld();
// Returns: "annotation_1234567890_abc123def"
```

sanitizeAnnotationText(text)

Sanitizes text for safe DOM insertion.

```
javascript
```

```
const clean = AnnotationModule.Utils.sanitizeAnnotationText(userInput);  
// Removes: <, >, ', "
```

parseAnnotationGeometry(annotationData)

Parses geometry from data string.

```
javascript  
  
const geometry = AnnotationModule.Utils.parseAnnotationGeometry("0.1,0.2,0.3,0.4,src");  
// Returns: {x: 0.1, y: 0.2, width: 0.3, height: 0.4, src: "src"}
```

Constants

AnnotationModule.Constants

DEFAULT_DIMENSIONS

```
javascript  
  
{  
  ... MIN_WIDTH: 300,  
  ... MIN_HEIGHT: 200,  
  ... MAX_PROCESSABLE_HEIGHT: 700,  
  ... DEFAULT_PADDING: 150,  
  ... RESIZE_PERCENTAGE: 20  
}
```

SELECTORS

```
javascript  
  
{  
  ... FIGURE_CLASS: '.fig',  
  ... GRAPHIC_IMAGE: '.graphic img',  
  ... COMMENT_SPAN: 'span[data-class="ckcommentsfull"]',  
  ... ANNOTATE_ATTR: 'data-annotate',  
  ... COMMENT_BOX_ATTR: 'data-user-comment-box',  
  ... USER_NAME_ATTR: 'data-user-name'  
}
```

UI_ELEMENTS

javascript

```
{
  ... IFRAME_ID: 'if1',
  ... IMAGE_ID: 'imgID',
  ... SPINNER_ID: 'spinner_dv'
}
```

EVENTS

javascript

```
{
  ... ANNOTATION_CREATED: 'onAnnotationCreated',
  ... ANNOTATION_UPDATED: 'onAnnotationUpdated',
  ... ANNOTATION_REMOVED: 'onAnnotationRemoved',
  ... BEFORE_ANNOTATION_REMOVED: 'beforeAnnotationRemoved',
  ... MOUSE_OVER_ANNOTATION: 'onMouseOverAnnotation'
}
```

Usage Examples

Basic Usage

javascript

```
// Initialize module
const annotationModule = new AnnotationModule('annotations', errorTracker);

// Initialize processing loop
annotationModule.initLoop();

// Show annotation interface
await annotationModule.showLoop();

// Setup dimensions (called automatically)
annotationModule.setupDimensions();

// Bind events to annotation library
annotationModule.bindAnnotationEvents(annotationLibrary);

// Reset when needed
annotationModule.reset();

// Close module
annotationModule.closeModule();
```

Advanced Configuration

```
javascript

// Custom options
const options = {
  ... customDimensions: true,
  ... debugMode: true
};

const module = new AnnotationModule('annotations', errorTracker, options);

// Show with custom settings
await module.showLoop({
  ... autoFocus: true,
  enableKeyboardShortcuts: true
});
```

Error Handling

```
javascript
```

```
try {  
... await annotationModule.showLoop();  
} catch (error) {  
  // Error is automatically logged via ErrorLogTrace  
... console.log('Annotation interface failed to load');  
}
```

Performance Considerations

Optimization Strategies

1. **Reference Caching:** Editor references cached in `cacheEditorReferences()`
2. **Lazy Loading:** Annotation interface loaded only when needed
3. **Efficient DOM Queries:** Selectors cached in state
4. **Event Cleanup:** Proper event listener removal
5. **Memory Management:** References cleared on reset/close

Performance Monitoring

```
javascript  
  
// Module provides built-in logging for performance tracking  
// Check console for timing information:  
// [AnnotationModule] Operation completed  
// [AnnotationModule] Frame dimensions applied successfully
```

Troubleshooting

Common Issues

Module Not Initializing

```
javascript
```

```
// Check prerequisites
if (!GlobalEditor) {
  ... console.error('GlobalEditor not available');
}

// Verify module state
const module = new AnnotationModule('test', null);
console.log('Module ready:', module.isModuleReady);
```

Image Not Found

javascript

```
// Ensure image is selected in editor
const selection = GlobalEditor.getSelection();
const element = selection.getStartElement();
console.log('Selected element:', element.getName());
```

Iframe Not Loading

javascript

```
// Check plugin path
const path = CKEDITOR.plugins.getPath('ImageAnotation_old');
console.log('Plugin path:', path);

// Verify iframe element exists
const iframe = document.getElementById('if1');
console.log('Iframe found:', !!iframe);
```

Debug Mode

javascript

```
// Enable detailed logging by ensuring debug.log is available
if (typeof debug !== 'undefined') {
  ... debug.log('Debug mode enabled');
}
```

Migration Guide

From v1.0 to v2.0

Method Name Changes

javascript

// Old → New

AssignVar_EventLoop() → assignVariablesEventLoop()

getSource() → getImageSource()

setUpWidthHeight() → setupDimensions()

pluginLoadScript() → loadPluginScript()

annotationEvtBind() → bindAnnotationEvents()

Property Changes

javascript

// Old → New

CUR_FIG → currentFigure

CUR_ELM → currentElement

CUR_ARR → currentArray

Async Methods

javascript

// v1.0 (callback-based)

module.showLoop(options, callback);

// v2.0 (promise-based)

await module.showLoop(options);

Best Practices

Development Guidelines

1. **Always await async methods:** `showLoop()` and dimension calculations
2. **Handle errors gracefully:** Use try-catch with async operations
3. **Reset after use:** Call `reset()` to clean up state
4. **Check prerequisites:** Validate dependencies before operations
5. **Use static utilities:** Leverage provided utility functions

Code Examples

javascript

//  *Good Practice*

```
try {  
  ... await annotationModule.showLoop();  
  // Success handling  
} catch (error) {  
  ... // Error automatically logged  
} finally {  
  ... annotationModule.reset();  
}
```

//  *Avoid*

```
annotationModule.showLoop(); // Missing await  
// No error handling or cleanup
```

Memory Management

javascript

// *Proper cleanup*

```
annotationModule.closeModule(); // Calls cleanup automatically
```

// *Manual cleanup if needed*

```
annotationModule.reset();  
annotationModule.cleanup();
```

API Reference Summary

Public Methods

- `initLoop()` - Initialize processing
- `showLoop(options)` - Show annotation interface
- `fireRestore()` - Restore state
- `reset()` - Reset to initial state
- `closeModule()` - Close and cleanup
- `setupDimensions()` - Setup responsive dimensions
- `loadPluginScript(library)` - Load annotation plugin
- `bindAnnotationEvents(library)` - Bind event handlers
- `getUrl()` - Get annotation interface URL

Static Utilities

- `AnnotationModule.Utils.validateAnnotationData()`
- `AnnotationModule.Utils.generateUniqueId()`
- `AnnotationModule.Utils.sanitizeAnnotationText()`
- `AnnotationModule.Utils.parseAnnotationGeometry()`

Constants

- `AnnotationModule.Constants.DEFAULT_DIMENSIONS`
- `AnnotationModule.Constants.SELECTORS`
- `AnnotationModule.Constants.UI_ELEMENTS`
- `AnnotationModule.Constants.EVENTS`